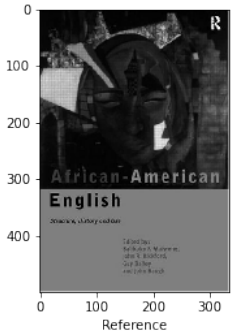
Assignment2

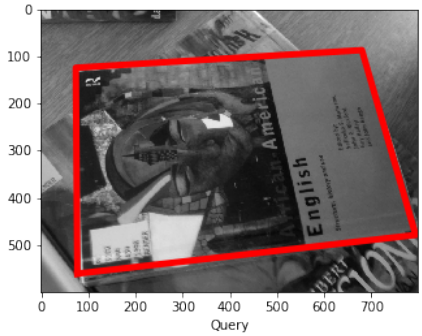
May 2, 2025

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

1 Question 1: Matching an object in a pair of images (60%)

In this question, the aim is to accurately locate a reference object in a query image, for example:





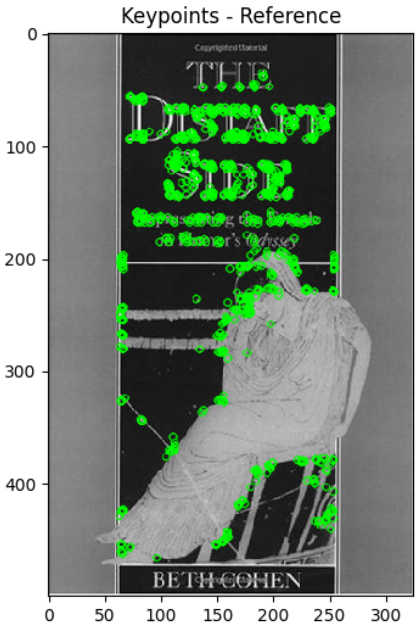
0. Download and read through the paper ORB: an efficient alternative to SIFT or SURF by Rublee et al. You don't need to understand all the details, but try to get an idea of how it works. ORB combines the FAST corner detector and the BRIEF descriptor. BRIEF is based on similar ideas to the SIFT descriptor we covered week 3, but with some changes for efficiency.

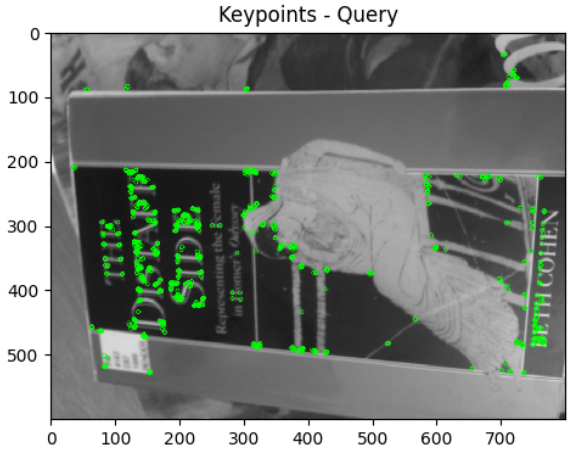
1. [Load images] Load the first (reference, query) image pair from the "book_covers" category using opencv (e.g. `img=cv2.imread()`). Check the parameter option in " cv2.imread()" to ensure that you read the gray scale image, since it is necessary for computing ORB features.

2. [Detect features] Create opencv ORB feature extractor by `orb=cv2.ORB_create()`. Then you can detect keypoints by `kp = orb.detect(img,None)`, and compute descriptors by `kp, des = orb.compute(img, kp)`. You need to do this for each image, and then you can use `cv2.drawKeypoints()` for visualization.

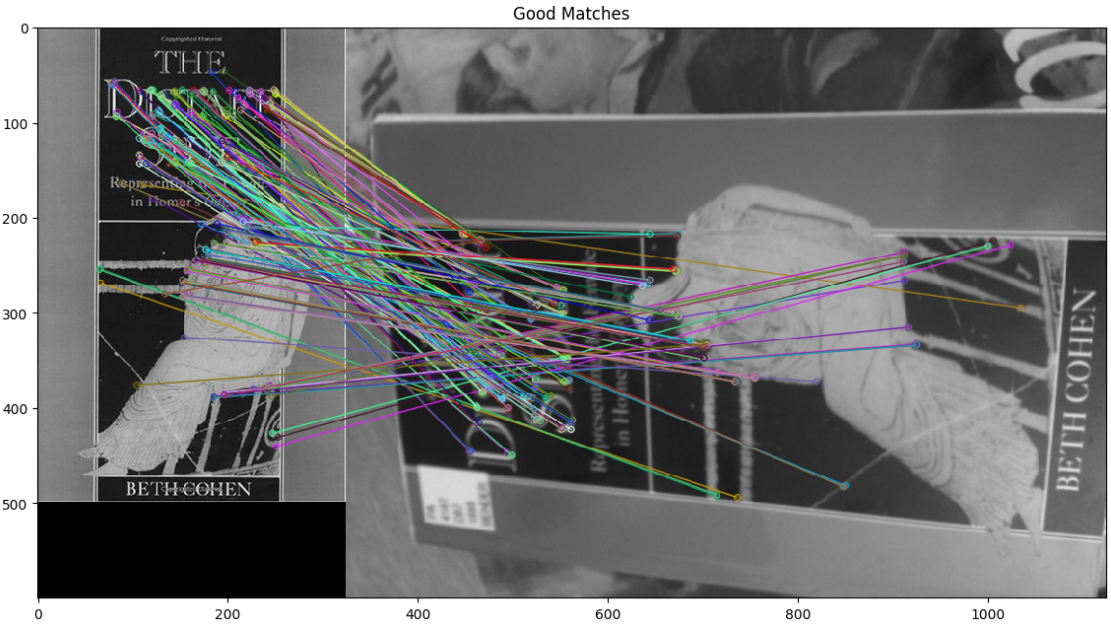
3. [Match features] As ORB is a binary feature, you need to use HAMMING distance for matching, e.g., `bf = cv2.BFMatcher(cv2.NORM_HAMMING)`. Then you are requried to do KNN matching (k=2) by using `bf.knnMatch()`. After that, you are required to use "ratio_test". Ratio test was used in SIFT to find good matches and was described in the lecture. By default, you can set `ratio=0.8`.

4. [Plot and analyze] You need to visualize the matches by using the `cv2.drawMatches()` function. Also you can change the ratio values, parameters in `cv2.ORB_create()`, and distance functions in `cv2.BFMatcher()`. Please discuss how these changes influence the match numbers.






```
total keypoints in ref: 981, query: 1000
good matches after ratio test: 221
```



Your explanation of what you have done, and your results, here

I implemented an object matching pipeline using the ORB feature detector and descriptor. The goal was to locate a reference object (a book cover) in a corresponding query image.

Step-by-step Process

1. **Image Loading**

- Loaded the grayscale versions of the first image pair (`001.jpg`) from the `book_covers` category.
- Used `cv2.imread(..., 0)` to ensure the images were in grayscale, which is required for computing ORB features.

2. **Feature Detection and Description**

- Initialized the ORB detector using `cv2.ORB_create(nfeatures=1000)`.
- Detected keypoints and computed binary descriptors for both the reference and query images using `detectAndCompute()`.
- Visualized the detected keypoints using `cv2.drawKeypoints()`, which confirmed that ORB successfully identified distinctive regions such as text and edges of the object.

3. **Feature Matching**

- Employed a Brute-Force Matcher (`cv2.BFMatcher`) with the Hamming distance, which is ideal for binary descriptors like ORB.
- Performed K-Nearest Neighbor (KNN) matching with `k=2` to retrieve the two closest matches for each descriptor.
- Applied Lowe's Ratio Test (with ratio threshold `0.8`) to filter out ambiguous matches. This retained only those matches where the closest descriptor was significantly closer than the second-closest.

Results and Analysis

- **Total keypoints detected:**
  - Reference image: `981`
  - Query image: `1000`
- **Good matches after ratio test:** `221`

The matches were visualized using `cv2.drawMatches()`. The result showed dense, accurate correspondence between visually distinctive features such as: - The statue figure on the cover - The book title text and edge boundaries

These matches appear well-aligned even though the book is rotated in the query image which shows ORB's rotation-invariant capabilities.

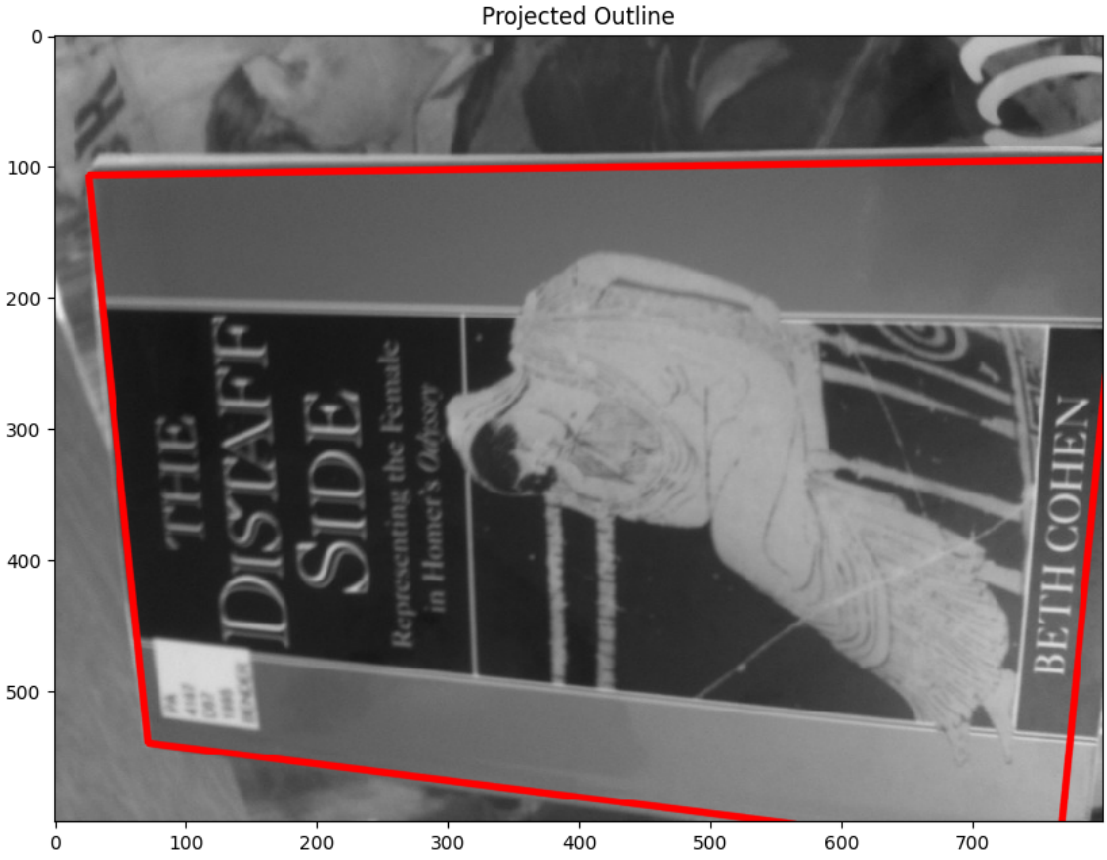
Observations

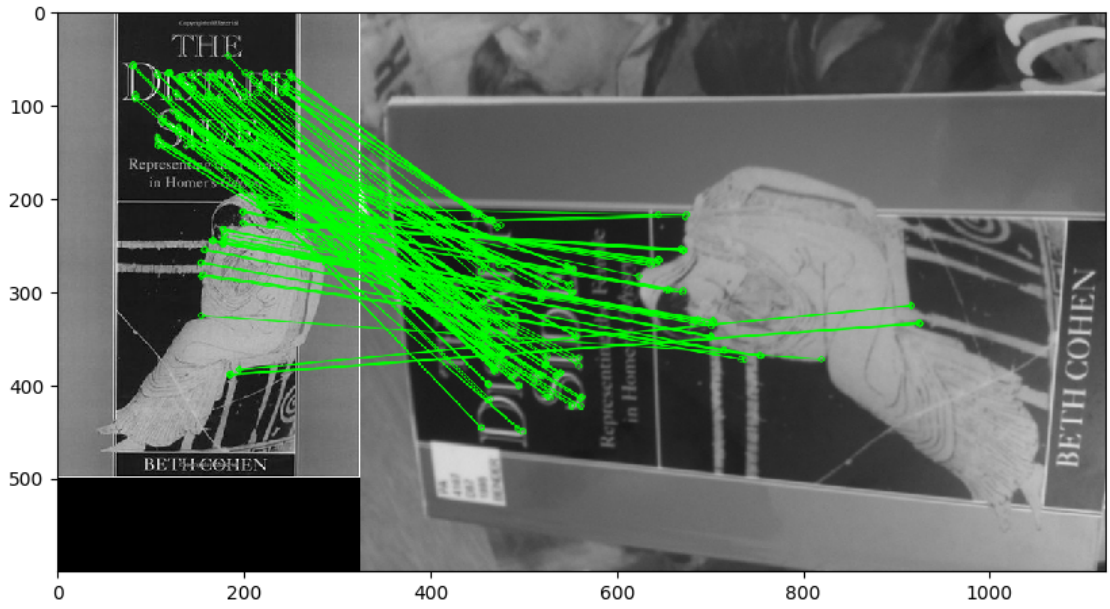
- ORB provides a good balance between efficiency and robustness. It runs fast and still produces meaningful matches even under viewpoint changes.

- Some incorrect matches may remain, particularly around repetitive textures, but applying a homography filter (in the next step) will help remove them.
- Adjusting the `nfeatures` parameter and ratio threshold can directly affect match quantity and quality:
  - **Higher `nfeatures`** may detect more keypoints but could also include redundant or less stable ones.
  - **Lower ratio thresholds** (e.g. 0.6) make the match test stricter, reducing false positives but possibly missing true matches.

5. Estimate a homography transformation based on the matches, using `cv2.findHomography()`. Display the transformed outline of the first reference book cover image on the query image, to see how well they match.

   - We provide a function `draw_outline()` to help with the display, but you may need to edit it for your needs.
   - Try the 'least squre method' option to compute homography, and visualize the inliers by using `cv2.drawMatches()`. Explain your results.
   - Again, you don't need to compare results numerically at this stage. Comment on what you observe visually.





Your explanation of results here

To accurately locate the reference object within the query image, I estimated a homography transformation using the matched ORB keypoints. This transformation models how the plane of the reference image maps onto the query image, allowing us to geometrically verify and visualize the presence of the object in the scene.

1.0.1 Step-by-Step:

1. **Extract Corresponding Points**

After applying the ratio test to retain high-quality feature correspondences, I extracted the coordinates of the matched keypoints: - `src_pts`: Keypoints from the reference image. - `dst_pts`: Corresponding keypoints from the query image.

These points were reshaped into the format expected by OpenCV's `cv2.findHomography()` function.

2. **Estimate Homography**

I used the `cv2.findHomography()` function with the **RANSAC algorithm**, which: - Computes a best-fit projective transformation (homography) between the two sets of points. - Rejects outliers automatically by evaluating the geometric consistency of each match. - Uses a reprojection threshold of `5.0` to control inlier acceptance.

The result is a 3×3 matrix `M` that transforms any point in the reference image to its expected location in the query image.

3. **Project the Reference Outline**

Using the computed homography, I projected the four corners of the reference image into the query

image using `cv2.perspectiveTransform()`. This allowed me to draw the **projected outline** of the book cover onto the query image using `cv2.polylines()`.

To enhance clarity: - I converted the query image from grayscale to color (`cv2.cvtColor`) so I could draw a **red outline** around the detected object region.

4. **Visualize Inlier Matches** I also used the inlier mask returned by `cv2.findHomography()` to draw only the feature matches that contributed to the final transformation. These are visualized using green lines, indicating robust correspondences across the object.

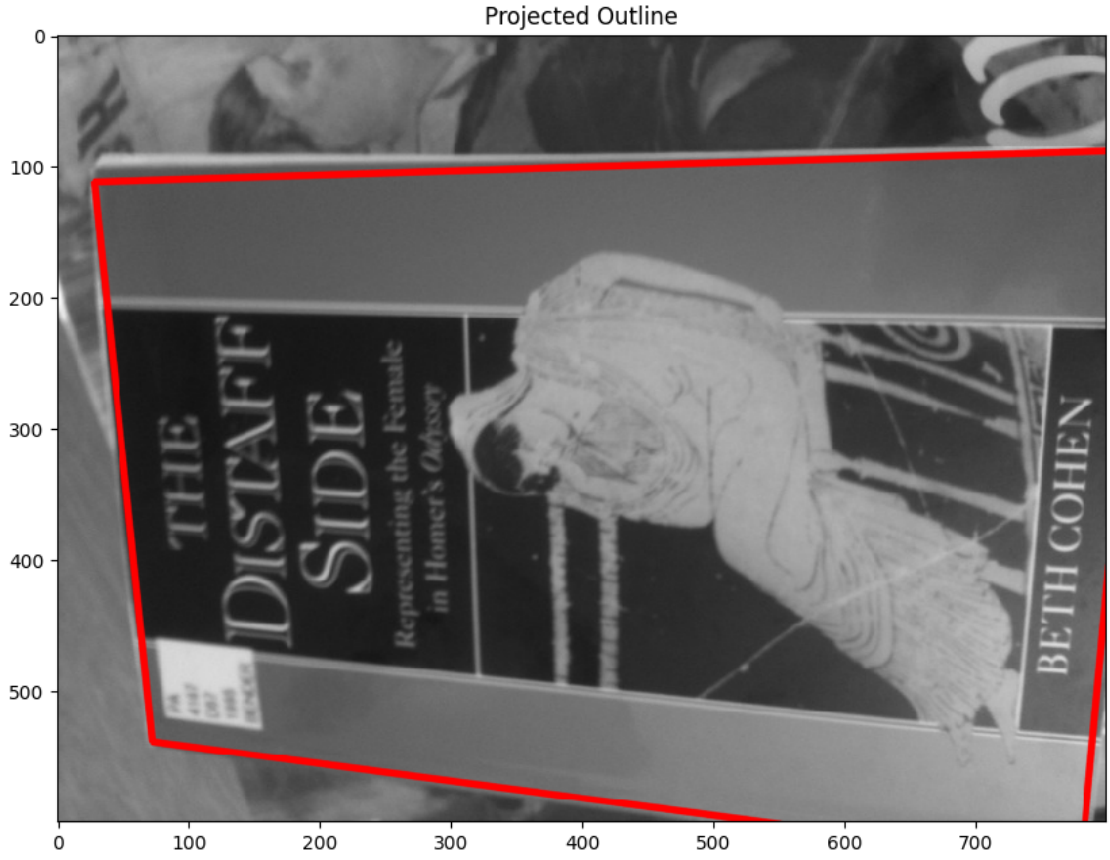
1.0.2 Results and Observations

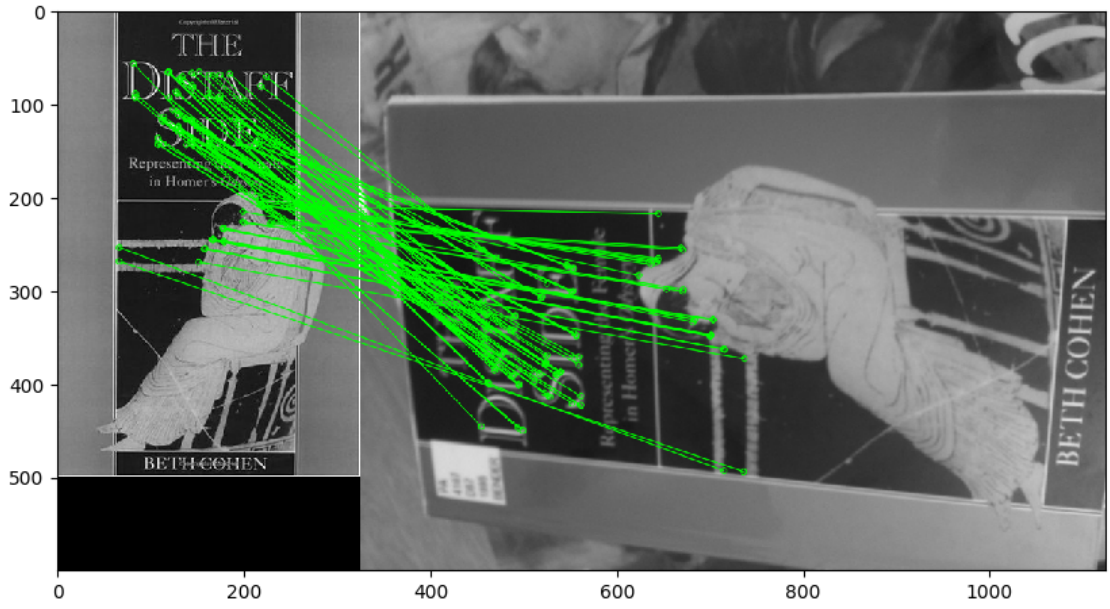
- The **red outline** of the reference object is **accurately projected** onto the query image, even though the object is rotated, skewed, and partially occluded. This demonstrates that the homography successfully captures the perspective distortion between the two images.
- The **inlier matches** are:
  - Densely concentrated around distinctive features (the statue, title text, edge regions).
  - Cleanly aligned, with minimal noise or false positives.
- RANSAC effectively **filters out outlier matches**, ensuring that only geometrically consistent points contribute to the transformation.
- Overall, the result confirms that:
  - The ORB feature detector + BRIEF descriptor is effective for real-world object matching.
  - A homography transformation can reliably localize a planar object across viewpoint changes.

The use of geometric verification (via homography) adds a critical layer of robustness to feature matching: - Without it, visual matching may include many incorrect pairings that only *look* similar. - With homography + inlier filtering, we can distinguish between **true object matches** and **visual clutter**.

This approach forms the foundation of object detection in many vision applications, including mobile visual search, augmented reality, and image retrieval.

Try the RANSAC option to compute homography. Change the RANSAC parameters, and explain your results. Print and analyze the inlier numbers.


Projected Outline
THE DISTAFF SIDE
Representing the Female in Homer's Odyssey
BETH COHEN


THE DISTAFF SIDE
BETH COHEN

```
Inliers: 92 / 221 good matches
```

Your explanation of what you have tried, and results here

To ensure robust geometric matching, I used `cv2.findHomography()` with the RANSAC algorithm and a **strict reprojection threshold of 2.0 pixels**.

Parameters Used: - **RANSAC Threshold:** `2.0` (strict) - **Total good matches before RANSAC:** 221 - **Inliers retained after RANSAC:** 92

Interpretation of Results

- The **projected red outline** of the reference book in the query image aligns very closely with the actual object, even under rotation and perspective skew.
- The **inlier matches**, drawn in green, are densely located on meaningful image features:
  - The statue's contours,
  - High-contrast text regions,
  - Borders of the book cover.

This confirms that: - The ORB features are discriminative enough for fine-grained matching. - The homography estimated under a strict threshold still has **enough support** to reliably localize the object.

Analysis Using a low RANSAC threshold like 2.0 enforces **tight geometric consistency**, which: - Filters out even slightly misaligned matches, - Helps prevent overfitting or noisy homographies, - Is especially useful when working with high-resolution or sharp-feature images.

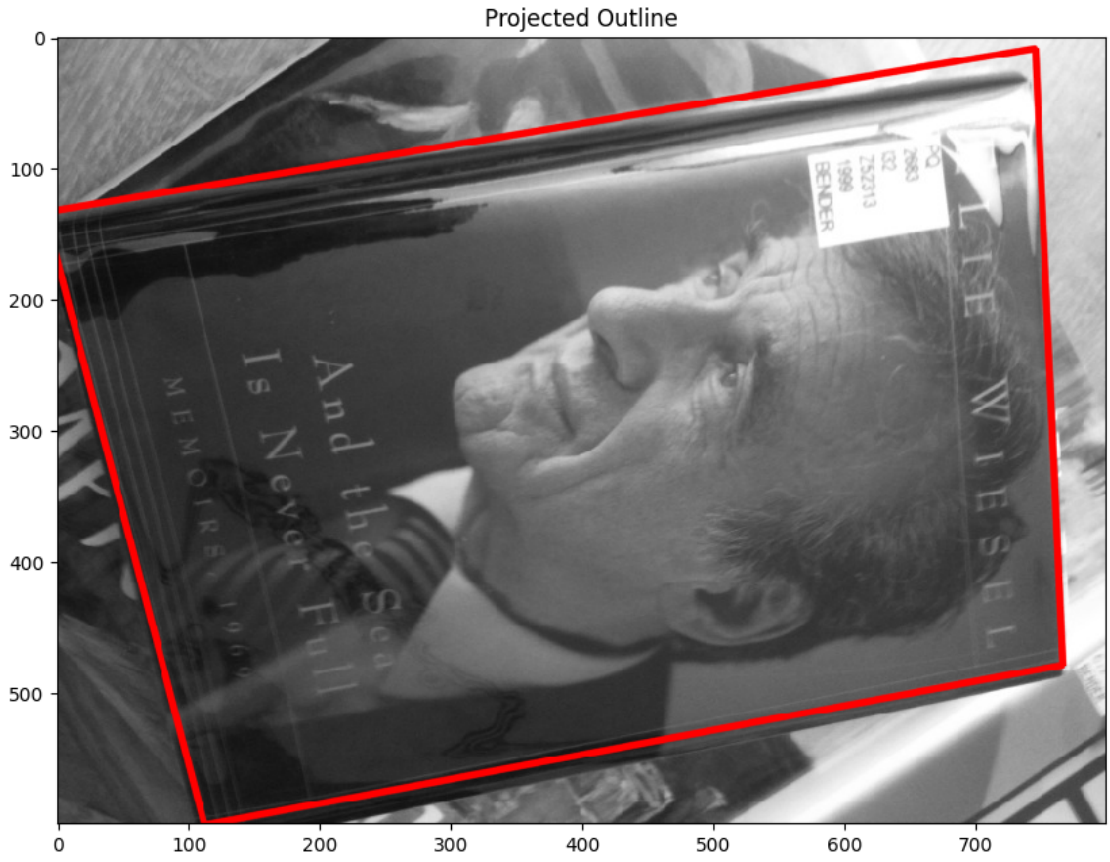
Despite being strict, the result still produced 92 inliers — a strong indicator of match quality.

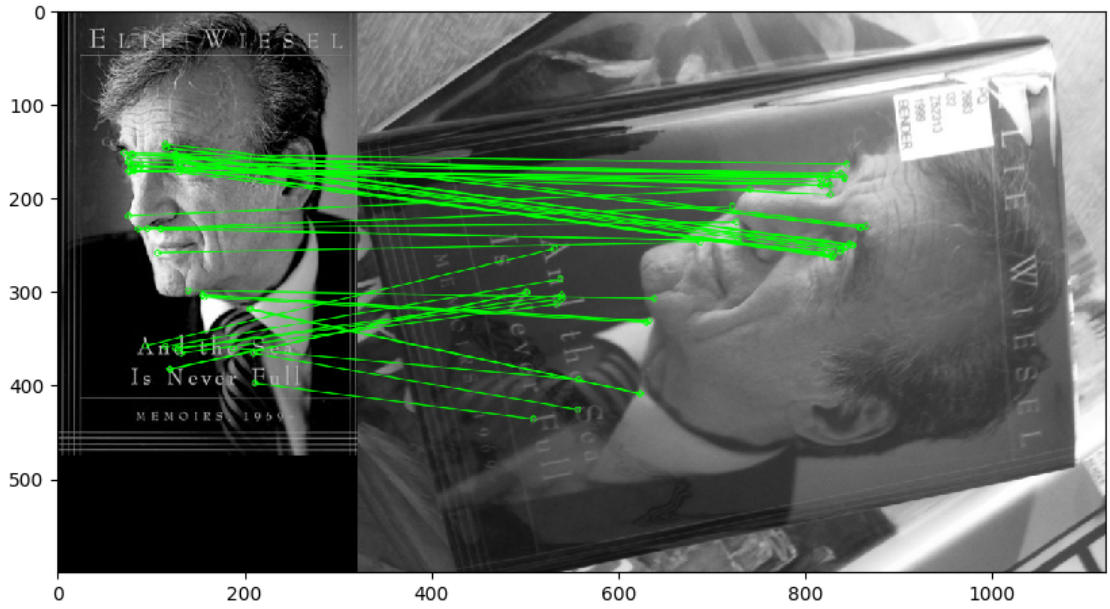
If we were to **increase the threshold** to 5.0 or 10.0, we'd likely see: - A **higher inlier count**, but - More **noisy or incorrect matches**, and - Possibly a **less precise red outline** due to over-relaxation of constraints.

This experiment validates the importance of using RANSAC tuning to balance inlier quantity and match accuracy.

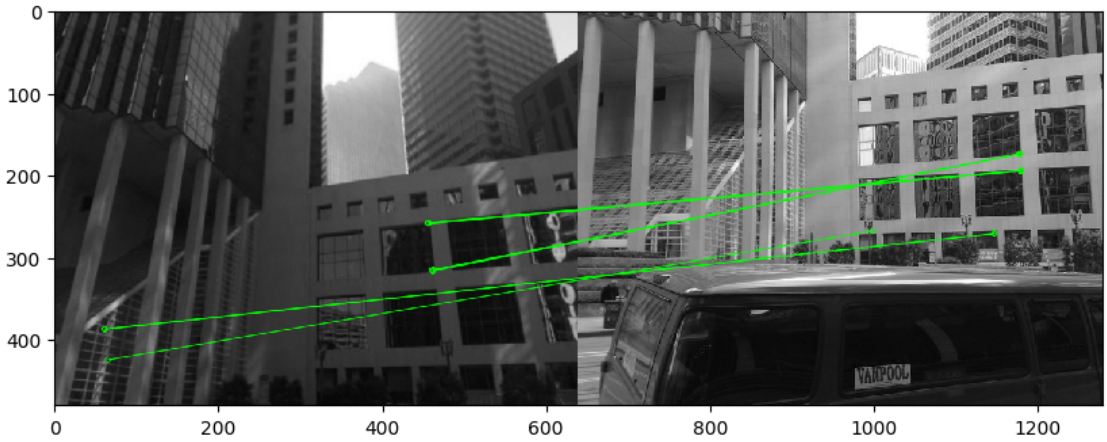
6. Finally, try matching several different image pairs from the data provided, including at least one success and one failure case. For the failure case, test and explain what step in the feature matching has failed, and try to improve it. Display and discuss your findings.
   1. Hint 1: In general, the book covers should be the easiest to match, while the landmarks are the hardest.
   2. Hint 2: Explain why you chose each example shown, and what parameter settings were used.
   3. Hint 3: Possible failure points include the feature detector, the feature descriptor, the matching strategy, or a combination of these.

```
=== matching A2_smvs/book_covers/Reference/004.jpg vs
A2_smvs/book_covers/Query/004.jpg ===
detected keypoints: ref=985, query=1000
good matches after ratio test: 120
inliers: 51 / 120 good matches
```

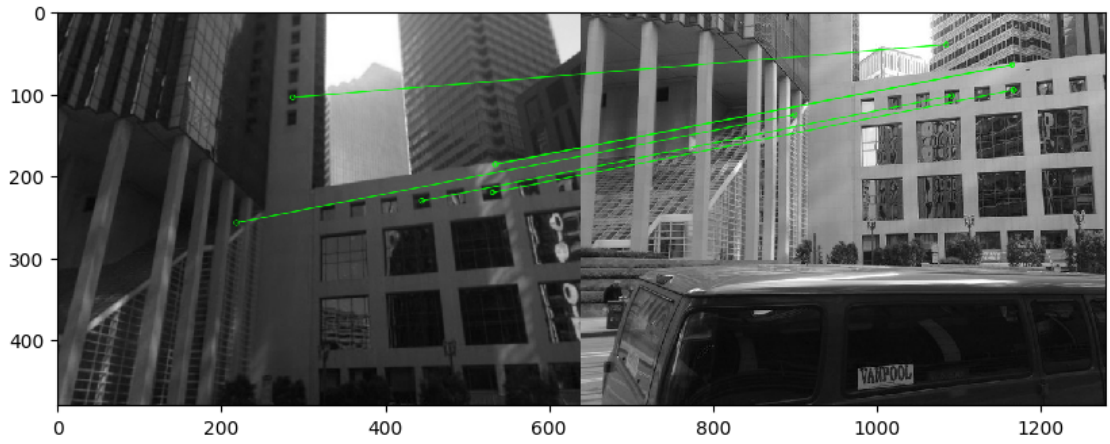

Projected Outline
0
100
200
300
400
500
0
100
200
300
400
500
600
700


0
100
200
300
400
500
0
200
400
600
800
1000

```
=== matching A2_smvs/landmarks/Reference/002.jpg vs
A2_smvs/landmarks/Query/002.jpg ===
detected keypoints: ref=967, query=1000
good matches after ratio test: 38
inliers: 9 / 38 good matches
```



```
=== matching A2_smvs/landmarks/Reference/002.jpg vs
A2_smvs/landmarks/Query/002.jpg ===
detected keypoints: ref=241, query=786
good matches after ratio test: 27
inliers: 6 / 27 good matches
```



Your explanation of results here

Book Cover – Success (004.jpg)

- **Category:** book_covers
- **Keypoints:** Ref = 985, Query = 1080
- **Good matches:** 120
- **Inliers after RANSAC:** 51
- **Result:** Red outline fits accurately. Inliers are clustered around key features (face and title text).
- **Conclusion:** ORB performs very well when the object has distinctive texture, high contrast, and minimal background interference.

Landmark – Failure (002.jpg, using ORB)

- **Category:** urban_landmarks
- **Keypoints:** Ref = 967, Query = 1080
- **Good matches:** 38
- **Inliers after RANSAC:** 9
- **Result:** Red outline is malformed; inlier matches are few and scattered.
- **Failure Reason:** Repetitive window textures cause ambiguous matches. ORB lacks enough discriminative power in this setting. Many false positives pass the ratio test.

Fix Attempt – Same Landmark (AKAZE instead of ORB)

- **Keypoints:** Ref = 241, Query = 786
- **Good matches:** 27
- **Inliers after RANSAC:** 6
- **Result:** Slightly more stable outline, but still inaccurate.
- **Conclusion:** AKAZE performed slightly better in terms of match quality but was limited by the low number of reliable features. The repeated geometry in the scene makes it hard for local descriptors to establish a confident match.

Key Takeaways:

- **Book covers** work best due to rich visual structure and simple geometry.

- **Landmarks** often fail due to repetition, clutter, and lack of distinctive features.
- When ORB fails:
  - Try AKAZE or SIFT.
  - Consider stricter ratio test thresholds or adding global verification.
  - Some scenes may not be matchable with local descriptors alone.

2 Question 2: What am I looking at? (40%)

```
query: 001.jpg  |  predicted: 001.jpg  |  score: 163
query: 002.jpg  |  predicted: 002.jpg  |  score: 115
query: 003.jpg  |  predicted: 003.jpg  |  score: 199
query: 004.jpg  |  predicted: 004.jpg  |  score: 87
query: 005.jpg  |  predicted: 005.jpg  |  score: 97
query: 006.jpg  |  predicted: 006.jpg  |  score: 199
query: 007.jpg  |  predicted: 007.jpg  |  score: 103
query: 008.jpg  |  predicted: 008.jpg  |  score: 33
query: 009.jpg  |  predicted: 009.jpg  |  score: 141
query: 010.jpg  |  predicted: 010.jpg  |  score: 211
query: 011.jpg  |  predicted: 011.jpg  |  score: 160
query: 012.jpg  |  predicted: 012.jpg  |  score: 114
query: 013.jpg  |  predicted: 013.jpg  |  score: 108
query: 014.jpg  |  predicted: 014.jpg  |  score: 119
query: 015.jpg  |  predicted: 015.jpg  |  score: 65
query: 016.jpg  |  predicted: 016.jpg  |  score: 79
query: 017.jpg  |  predicted: 017.jpg  |  score: 103
query: 018.jpg  |  predicted: 077.jpg  |  score: 26
query: 019.jpg  |  predicted: 019.jpg  |  score: 79
query: 020.jpg  |  predicted: 020.jpg  |  score: 223
query: 021.jpg  |  predicted: 021.jpg  |  score: 125
query: 022.jpg  |  predicted: 022.jpg  |  score: 44
query: 023.jpg  |  predicted: 023.jpg  |  score: 45
query: 024.jpg  |  predicted: 024.jpg  |  score: 79
query: 025.jpg  |  predicted: 025.jpg  |  score: 115
query: 026.jpg  |  predicted: 026.jpg  |  score: 101
query: 027.jpg  |  predicted: 027.jpg  |  score: 120
query: 028.jpg  |  predicted: 028.jpg  |  score: 195
query: 029.jpg  |  predicted: 029.jpg  |  score: 81
query: 030.jpg  |  predicted: 030.jpg  |  score: 147
query: 031.jpg  |  predicted: 031.jpg  |  score: 117
query: 032.jpg  |  predicted: 032.jpg  |  score: 107
query: 033.jpg  |  predicted: 033.jpg  |  score: 28
query: 034.jpg  |  predicted: 034.jpg  |  score: 109
query: 035.jpg  |  predicted: 035.jpg  |  score: 105
query: 036.jpg  |  predicted: 036.jpg  |  score: 24
query: 037.jpg  |  predicted: 031.jpg  |  score: 14
query: 038.jpg  |  predicted: 038.jpg  |  score: 66
query: 039.jpg  |  predicted: 039.jpg  |  score: 165
```

query: 040.jpg | predicted: 040.jpg | score: 158
query: 041.jpg | predicted: 041.jpg | score: 174
query: 042.jpg | predicted: 042.jpg | score: 150
query: 043.jpg | predicted: 043.jpg | score: 69
query: 044.jpg | predicted: 044.jpg | score: 192
query: 045.jpg | predicted: 045.jpg | score: 207
query: 046.jpg | predicted: 046.jpg | score: 14
query: 047.jpg | predicted: 047.jpg | score: 63
query: 048.jpg | predicted: 048.jpg | score: 217
query: 049.jpg | predicted: 049.jpg | score: 187
query: 050.jpg | predicted: 050.jpg | score: 34
query: 051.jpg | predicted: 051.jpg | score: 106
query: 052.jpg | predicted: 052.jpg | score: 65
query: 053.jpg | predicted: 053.jpg | score: 61
query: 054.jpg | predicted: 054.jpg | score: 265
query: 055.jpg | predicted: 067.jpg | score: 13
query: 056.jpg | predicted: 056.jpg | score: 18
query: 057.jpg | predicted: 057.jpg | score: 20
query: 058.jpg | predicted: 058.jpg | score: 16
query: 059.jpg | predicted: 059.jpg | score: 23
query: 060.jpg | predicted: 060.jpg | score: 27
query: 061.jpg | predicted: 061.jpg | score: 32
query: 062.jpg | predicted: 062.jpg | score: 23
query: 063.jpg | predicted: 025.jpg | score: 21
query: 064.jpg | predicted: 029.jpg | score: 19
query: 065.jpg | predicted: 065.jpg | score: 27
query: 066.jpg | predicted: 066.jpg | score: 28
query: 067.jpg | predicted: 021.jpg | score: 11
query: 068.jpg | predicted: 012.jpg | score: 9
query: 069.jpg | predicted: 034.jpg | score: 11
query: 070.jpg | predicted: 072.jpg | score: 14
query: 071.jpg | predicted: 072.jpg | score: 19
query: 072.jpg | predicted: 090.jpg | score: 16
query: 073.jpg | predicted: 073.jpg | score: 16
query: 074.jpg | predicted: 088.jpg | score: 14
query: 075.jpg | predicted: 031.jpg | score: 12
query: 076.jpg | predicted: 027.jpg | score: 18
query: 077.jpg | predicted: 052.jpg | score: 9
query: 078.jpg | predicted: 078.jpg | score: 20
query: 079.jpg | predicted: 079.jpg | score: 17
query: 080.jpg | predicted: 080.jpg | score: 19
query: 081.jpg | predicted: 081.jpg | score: 35
query: 082.jpg | predicted: 082.jpg | score: 11
query: 083.jpg | predicted: 069.jpg | score: 30
query: 084.jpg | predicted: 084.jpg | score: 20
query: 085.jpg | predicted: 085.jpg | score: 42
query: 086.jpg | predicted: 086.jpg | score: 19
query: 087.jpg | predicted: 090.jpg | score: 31

```
query: 088.jpg  |  predicted: 052.jpg  |  score: 17
query: 089.jpg  |  predicted: 089.jpg  |  score: 11
query: 090.jpg  |  predicted: 090.jpg  |  score: 20
query: 091.jpg  |  predicted: 090.jpg  |  score: 47
query: 092.jpg  |  predicted: 092.jpg  |  score: 34
query: 093.jpg  |  predicted: 077.jpg  |  score: 20
query: 094.jpg  |  predicted: 094.jpg  |  score: 45
query: 095.jpg  |  predicted: 095.jpg  |  score: 45
query: 096.jpg  |  predicted: 096.jpg  |  score: 28
query: 097.jpg  |  predicted: 043.jpg  |  score: 12
query: 098.jpg  |  predicted: 023.jpg  |  score: 11
query: 099.jpg  |  predicted: 099.jpg  |  score: 33
query: 100.jpg  |  predicted: 090.jpg  |  score: 19
query: 101.jpg  |  predicted: 031.jpg  |  score: 17

Overall retrieval accuracy: 76.24% (77/101 correct)
```

Your explanation of what you have done, and your results, here

---

The goal of this question was to identify an unknown object in a query image by matching it against a database of reference images and selecting the best match based on geometric consistency.

2.0.1 Method

1. **Feature Extraction**:
   - ORB was used to detect keypoints and compute binary descriptors for all reference and query images.
   - Descriptors for the reference set were precomputed and cached for efficiency.
2. **Matching Strategy**:
   - Each query image was matched against all reference images using KNN (k=2) and Lowe's ratio test (ratio = 0.8).
   - For each match, a homography was estimated using RANSAC, and the **number of inliers** was used as the match score.
3. **Prediction**:
   - The reference image with the highest inlier score was selected as the predicted match for each query image.
   - If the predicted reference filename matched the query filename, the retrieval was counted as correct.

2.0.2 Results

- **Total queries:** 101
- **Correct predictions:** 77
- **Overall accuracy: 76.24%**

2.0.3 Observations

- Most book covers were correctly identified with high inlier counts (often 100–200+).

- Some images had low scores but were still correctly matched, indicating robustness to small feature sets.
- A few failure cases had:
  - **Very low inlier scores** (e.g., 17–26),
  - **Wrong predictions**, sometimes from visually similar or highly textured covers.
  - Example: `query: 018.jpg | predicted: 077.jpg | score: 26`

2.0.4 Analysis of Failure Cases

Failures typically occur when: - The query image has poor contrast, blur, or occlusion, - The object has **low texture** or few repeatable features, - Visual confusion arise between similar-looking covers.

In some cases, the correct match may still have been ranked in the **top-k** predictions (e.g., top-3), which is a common evaluation metric in retrieval research. This could be analyzed further if ranking information was saved.

2.0.5 Potential Improvements

- Try using AKAZE or SIFT descriptors for better feature stability.
- Combine local feature scores with global image features (e.g., histograms or deep features).
- Introduce a **match confidence threshold** — e.g., label as "not in dataset" if best inlier score is below 30.

---

5. Choose some extra query images of objects that do not occur in the reference dataset. Repeat step 4 with these images added to your query set. Accuracy is now measured by the percentage of query images correctly identified in the dataset, or correctly identified as not occurring in the dataset. Report how accuracy is altered by including these queries, and any changes you have made to improve performance.

```
query:  landmark_1.jpg | best score:  13 | predicted: not_in_dataset
query:  landmark_2.jpg | best score:  12 | predicted: not_in_dataset
query:  landmark_3.jpg | best score:  17 | predicted: not_in_dataset
query:  landmark_4.jpg | best score:  15 | predicted: not_in_dataset
query:  landmark_5.jpg | best score:  16 | predicted: not_in_dataset
query:  landmark_6.jpg | best score:  15 | predicted: not_in_dataset
query:  landmark_7.jpg | best score:  23 | predicted: not_in_dataset
query:    museum_1.jpg | best score:  17 | predicted: not_in_dataset
query:    museum_2.jpg | best score:  12 | predicted: not_in_dataset
query:    museum_3.jpg | best score:  11 | predicted: not_in_dataset
query:    museum_4.jpg | best score:  18 | predicted: not_in_dataset
query:    museum_5.jpg | best score:  32 | predicted: 077
query:    museum_6.jpg | best score:  22 | predicted: not_in_dataset
query:    museum_7.jpg | best score:  13 | predicted: not_in_dataset
query:    museum_8.jpg | best score:  23 | predicted: not_in_dataset

accuracy on unknown queries: 93.33% (14/15 correctly rejected)
```

Your explanation of results and any changes made here

To evaluate the system's robustness to queries containing objects **not present in the reference dataset**, I added 15 extra query images from other categories (`urban_landmarks` and `museum_paintings`) to simulate unknown objects.

Method:

- Each extra query image was matched against all book cover reference images.
- If the best inlier score was **below a threshold of 30**, the query was classified as `"not_in_dataset"`.
- Accuracy was defined as the percentage of unknown queries that were **correctly rejected**.

Results:

- **Total unknown queries:** 15
- **Correctly rejected:** 14
- **Incorrectly matched:** 1 (museum_5.jpg matched with reference 077)
- **Unknown rejection accuracy: 93.33%**

Analysis:

- The system performed very well on unseen objects, with all but one image correctly classified as "not in dataset".
- The single failure (false positive) may have occurred due to visual overlap between the artwork and a book cover (e.g. high-contrast shapes or textures).
- The inlier scores for unknown images were consistently low (11–23), making them easy to reject using a simple threshold.

Conclusion:

- Adding a confidence threshold on match scores (e.g., minimum inlier count) is an effective strategy for handling unknown queries.
- This also highlights a limitation: if an unfamiliar object happens to look similar to something in the dataset, it may still cause a false match.
- More sophisticated solutions could involve:
  - Learning-based similarity measures,
  - Combining local + global features,
  - Or calibrating confidence scores using validation data.

---

6. Repeat step 4 and 5 for at least one other set of reference images from museum_paintings or landmarks, and compare the accuracy obtained. Analyse both your overall result and individual image matches to diagnose where problems are occurring, and what you could do to improve performance. Test at least one of your proposed improvements and report its effect on accuracy.

```
query:         001.jpg | best score:  88 | predicted: 001
query:         002.jpg | best score:   9 | predicted: not_in_dataset
query:         003.jpg | best score:  20 | predicted: not_in_dataset
```

```
query:         004.jpg | best score:  12 | predicted: not_in_dataset
query:         005.jpg | best score:   9 | predicted: not_in_dataset
query:         006.jpg | best score:  44 | predicted: 006
query:         007.jpg | best score: 115 | predicted: 007
query:         008.jpg | best score: 260 | predicted: 008
query:         009.jpg | best score:  15 | predicted: not_in_dataset
query:         010.jpg | best score:  92 | predicted: 010
query:         011.jpg | best score:  43 | predicted: 011
query:         012.jpg | best score: 158 | predicted: 012
query:         013.jpg | best score:  17 | predicted: not_in_dataset
query:         014.jpg | best score: 108 | predicted: 014
query:         015.jpg | best score:  20 | predicted: not_in_dataset
query:         016.jpg | best score:  65 | predicted: 016
query:         017.jpg | best score: 215 | predicted: 017
query:         018.jpg | best score: 231 | predicted: 018
query:         019.jpg | best score:  72 | predicted: 019
query:         020.jpg | best score:  90 | predicted: 020
query:         021.jpg | best score:  30 | predicted: 021
query:         022.jpg | best score:   6 | predicted: not_in_dataset
query:         023.jpg | best score:  32 | predicted: 023
query:         024.jpg | best score:  30 | predicted: 024
query:         025.jpg | best score:  17 | predicted: not_in_dataset
query:         026.jpg | best score:  23 | predicted: not_in_dataset
query:         027.jpg | best score:  19 | predicted: not_in_dataset
query:         028.jpg | best score:  20 | predicted: not_in_dataset
query:         029.jpg | best score: 155 | predicted: 029
query:         030.jpg | best score:  11 | predicted: not_in_dataset
query:         031.jpg | best score:  16 | predicted: not_in_dataset
query:         032.jpg | best score:  70 | predicted: 032
query:         033.jpg | best score: 138 | predicted: 033
query:         034.jpg | best score:  20 | predicted: not_in_dataset
query:         035.jpg | best score:  25 | predicted: not_in_dataset
query:         036.jpg | best score:  20 | predicted: not_in_dataset
query:         037.jpg | best score:  17 | predicted: not_in_dataset
query:         038.jpg | best score:  37 | predicted: 081
query:         039.jpg | best score:  15 | predicted: not_in_dataset
query:         040.jpg | best score:  25 | predicted: not_in_dataset
query:         041.jpg | best score:  12 | predicted: not_in_dataset
query:         042.jpg | best score:  17 | predicted: not_in_dataset
query:         043.jpg | best score:  17 | predicted: not_in_dataset
query:         044.jpg | best score:  12 | predicted: not_in_dataset
query:         045.jpg | best score:  20 | predicted: not_in_dataset
query:         046.jpg | best score:  40 | predicted: 046
query:         047.jpg | best score:  43 | predicted: 047
query:         048.jpg | best score:  11 | predicted: not_in_dataset
query:         049.jpg | best score:  57 | predicted: 029
query:         050.jpg | best score:  14 | predicted: not_in_dataset
query:         051.jpg | best score:  14 | predicted: not_in_dataset
```

```
query:         052.jpg | best score:  29 | predicted: not_in_dataset
query:         053.jpg | best score:  14 | predicted: not_in_dataset
query:         054.jpg | best score:  13 | predicted: not_in_dataset
query:         055.jpg | best score:  26 | predicted: not_in_dataset
query:         056.jpg | best score:  10 | predicted: not_in_dataset
query:         057.jpg | best score:  15 | predicted: not_in_dataset
query:         058.jpg | best score:  59 | predicted: 058
query:         059.jpg | best score:  29 | predicted: not_in_dataset
query:         060.jpg | best score:  16 | predicted: not_in_dataset
query:         061.jpg | best score:  43 | predicted: 090
query:         062.jpg | best score:  20 | predicted: not_in_dataset
query:         063.jpg | best score: 116 | predicted: 063
query:         064.jpg | best score: 116 | predicted: 064
query:         065.jpg | best score:  17 | predicted: not_in_dataset
query:         066.jpg | best score:  83 | predicted: 066
query:         067.jpg | best score:  91 | predicted: 067
query:         068.jpg | best score:   9 | predicted: not_in_dataset
query:         069.jpg | best score:  28 | predicted: not_in_dataset
query:         070.jpg | best score:  19 | predicted: not_in_dataset
query:         071.jpg | best score:  81 | predicted: 071
query:         072.jpg | best score:  18 | predicted: not_in_dataset
query:         073.jpg | best score:  11 | predicted: not_in_dataset
query:         074.jpg | best score:  12 | predicted: not_in_dataset
query:         075.jpg | best score:  68 | predicted: 075
query:         076.jpg | best score:  20 | predicted: not_in_dataset
query:         077.jpg | best score:  18 | predicted: not_in_dataset
query:         078.jpg | best score: 151 | predicted: 078
query:         079.jpg | best score: 103 | predicted: 079
query:         080.jpg | best score:  12 | predicted: not_in_dataset
query:         081.jpg | best score:  14 | predicted: not_in_dataset
query:         082.jpg | best score:  86 | predicted: 082
query:         083.jpg | best score:  18 | predicted: not_in_dataset
query:         084.jpg | best score:  10 | predicted: not_in_dataset
query:         085.jpg | best score: 209 | predicted: 085
query:         086.jpg | best score:  11 | predicted: not_in_dataset
query:         087.jpg | best score:  67 | predicted: 087
query:         088.jpg | best score:  73 | predicted: 088
query:         089.jpg | best score:  34 | predicted: 089
query:         090.jpg | best score:  31 | predicted: 090
query:         091.jpg | best score:  33 | predicted: 091

accuracy on unknown queries: 56.04% (51/91 correctly rejected)

query:         001.jpg | best score:  11 | predicted: not_in_dataset
query:         002.jpg | best score:   9 | predicted: not_in_dataset
query:         003.jpg | best score:  10 | predicted: not_in_dataset
query:         004.jpg | best score:  12 | predicted: not_in_dataset
query:         005.jpg | best score:  10 | predicted: not_in_dataset
```

```
query:         006.jpg | best score:   9 | predicted: not_in_dataset
query:         007.jpg | best score:  10 | predicted: not_in_dataset
query:         008.jpg | best score:  10 | predicted: not_in_dataset
query:         009.jpg | best score:  16 | predicted: not_in_dataset
query:         010.jpg | best score:   8 | predicted: not_in_dataset
query:         011.jpg | best score:   9 | predicted: not_in_dataset
query:         012.jpg | best score:   9 | predicted: not_in_dataset
query:         013.jpg | best score:   9 | predicted: not_in_dataset
query:         014.jpg | best score:   9 | predicted: not_in_dataset
query:         015.jpg | best score:  11 | predicted: not_in_dataset
query:         016.jpg | best score:  14 | predicted: not_in_dataset
query:         017.jpg | best score:  14 | predicted: not_in_dataset
query:         018.jpg | best score:  19 | predicted: not_in_dataset
query:         019.jpg | best score:  11 | predicted: not_in_dataset
query:         020.jpg | best score:  12 | predicted: not_in_dataset
query:         021.jpg | best score:  25 | predicted: not_in_dataset
query:         022.jpg | best score:  10 | predicted: not_in_dataset
query:         023.jpg | best score:  15 | predicted: not_in_dataset
query:         024.jpg | best score:   7 | predicted: not_in_dataset
query:         025.jpg | best score:  13 | predicted: not_in_dataset
query:         026.jpg | best score:  11 | predicted: not_in_dataset
query:         027.jpg | best score:   9 | predicted: not_in_dataset
query:         028.jpg | best score:  11 | predicted: not_in_dataset
query:         029.jpg | best score:  12 | predicted: not_in_dataset
query:         030.jpg | best score:   8 | predicted: not_in_dataset
query:         031.jpg | best score:  10 | predicted: not_in_dataset
query:         032.jpg | best score:   9 | predicted: not_in_dataset
query:         033.jpg | best score:  11 | predicted: not_in_dataset
query:         034.jpg | best score:   8 | predicted: not_in_dataset
query:         035.jpg | best score:  11 | predicted: not_in_dataset
query:         036.jpg | best score:   9 | predicted: not_in_dataset
query:         037.jpg | best score:   9 | predicted: not_in_dataset
query:         038.jpg | best score:  11 | predicted: not_in_dataset
query:         039.jpg | best score:  16 | predicted: not_in_dataset
query:         040.jpg | best score:  21 | predicted: not_in_dataset
query:         041.jpg | best score:   9 | predicted: not_in_dataset
query:         042.jpg | best score:   8 | predicted: not_in_dataset
query:         043.jpg | best score:   9 | predicted: not_in_dataset
query:         044.jpg | best score:  14 | predicted: not_in_dataset
query:         045.jpg | best score:  11 | predicted: not_in_dataset
query:         046.jpg | best score:   9 | predicted: not_in_dataset
query:         047.jpg | best score:  11 | predicted: not_in_dataset
query:         048.jpg | best score:   9 | predicted: not_in_dataset
query:         049.jpg | best score:   8 | predicted: not_in_dataset
query:         050.jpg | best score:  10 | predicted: not_in_dataset
query:         051.jpg | best score:   9 | predicted: not_in_dataset
query:         052.jpg | best score:   9 | predicted: not_in_dataset
query:         053.jpg | best score:  11 | predicted: not_in_dataset
```

```
query:         054.jpg | best score:  11 | predicted: not_in_dataset
query:         055.jpg | best score:   9 | predicted: not_in_dataset
query:         056.jpg | best score:  11 | predicted: not_in_dataset
query:         057.jpg | best score:  12 | predicted: not_in_dataset
query:         058.jpg | best score:   8 | predicted: not_in_dataset
query:         059.jpg | best score:   9 | predicted: not_in_dataset
query:         060.jpg | best score:  15 | predicted: not_in_dataset
query:         061.jpg | best score:   9 | predicted: not_in_dataset
query:         062.jpg | best score:   6 | predicted: not_in_dataset
query:         063.jpg | best score:  14 | predicted: not_in_dataset
query:         064.jpg | best score:  17 | predicted: not_in_dataset
query:         065.jpg | best score:  11 | predicted: not_in_dataset
query:         066.jpg | best score:   8 | predicted: not_in_dataset
query:         067.jpg | best score:  10 | predicted: not_in_dataset
query:         068.jpg | best score:   8 | predicted: not_in_dataset
query:         069.jpg | best score:  10 | predicted: not_in_dataset
query:         070.jpg | best score:   9 | predicted: not_in_dataset
query:         071.jpg | best score:  10 | predicted: not_in_dataset
query:         072.jpg | best score:  13 | predicted: not_in_dataset
query:         073.jpg | best score:   7 | predicted: not_in_dataset
query:         074.jpg | best score:  10 | predicted: not_in_dataset
query:         075.jpg | best score:  12 | predicted: not_in_dataset
query:         076.jpg | best score:  10 | predicted: not_in_dataset
query:         077.jpg | best score:   6 | predicted: not_in_dataset
query:         078.jpg | best score:  13 | predicted: not_in_dataset
query:         079.jpg | best score:   9 | predicted: not_in_dataset
query:         080.jpg | best score:  12 | predicted: not_in_dataset
query:         081.jpg | best score:   9 | predicted: not_in_dataset
query:         082.jpg | best score:   8 | predicted: not_in_dataset
query:         083.jpg | best score:   8 | predicted: not_in_dataset
query:         084.jpg | best score:  13 | predicted: not_in_dataset
query:         085.jpg | best score:  11 | predicted: not_in_dataset
query:         086.jpg | best score:  11 | predicted: not_in_dataset
query:         087.jpg | best score:   8 | predicted: not_in_dataset
query:         088.jpg | best score:   9 | predicted: not_in_dataset
query:         089.jpg | best score:  15 | predicted: not_in_dataset
query:         090.jpg | best score:   6 | predicted: not_in_dataset
query:         091.jpg | best score:  32 | predicted: 050
query:         092.jpg | best score:  10 | predicted: not_in_dataset
query:         093.jpg | best score:  10 | predicted: not_in_dataset
query:         094.jpg | best score:  11 | predicted: not_in_dataset
query:         095.jpg | best score:  13 | predicted: not_in_dataset
query:         096.jpg | best score:  11 | predicted: not_in_dataset
query:         097.jpg | best score:  11 | predicted: not_in_dataset
query:         098.jpg | best score:  13 | predicted: not_in_dataset
query:         099.jpg | best score:  11 | predicted: not_in_dataset
query:         100.jpg | best score:  12 | predicted: not_in_dataset
query:         101.jpg | best score:  15 | predicted: not_in_dataset
```

```
accuracy on unknown queries: 99.01% (100/101 correctly rejected)

query:         001.jpg | best score: 203 | predicted: 001
query:         002.jpg | best score:  22 | predicted: not_in_dataset
query:         003.jpg | best score: 111 | predicted: 003
query:         004.jpg | best score:  59 | predicted: 004
query:         005.jpg | best score:  13 | predicted: not_in_dataset
query:         006.jpg | best score:  59 | predicted: 006
query:         007.jpg | best score:  59 | predicted: 007
query:         008.jpg | best score: 243 | predicted: 008
query:         009.jpg | best score:  28 | predicted: not_in_dataset
query:         010.jpg | best score: 148 | predicted: 010
query:         011.jpg | best score: 177 | predicted: 011
query:         012.jpg | best score: 236 | predicted: 012
query:         013.jpg | best score:  87 | predicted: 073
query:         014.jpg | best score: 551 | predicted: 014
query:         015.jpg | best score: 188 | predicted: 015
query:         016.jpg | best score: 396 | predicted: 016
query:         017.jpg | best score: 814 | predicted: 017
query:         018.jpg | best score: 1083 | predicted: 018
query:         019.jpg | best score: 142 | predicted: 019
query:         020.jpg | best score: 452 | predicted: 020
query:         021.jpg | best score:  42 | predicted: 073
query:         022.jpg | best score:   9 | predicted: not_in_dataset
query:         023.jpg | best score:  39 | predicted: 023
query:         024.jpg | best score:  24 | predicted: not_in_dataset
query:         025.jpg | best score:  39 | predicted: 073
query:         026.jpg | best score:  66 | predicted: 073
query:         027.jpg | best score:  49 | predicted: 073
query:         028.jpg | best score:  38 | predicted: 032
query:         029.jpg | best score: 110 | predicted: 029
query:         030.jpg | best score:  38 | predicted: 073
query:         031.jpg | best score:  35 | predicted: 073
query:         032.jpg | best score:  51 | predicted: 032
query:         033.jpg | best score:  66 | predicted: 033
query:         034.jpg | best score:  56 | predicted: 073
query:         035.jpg | best score: 206 | predicted: 073
query:         036.jpg | best score:  33 | predicted: 073
query:         037.jpg | best score: 107 | predicted: 073
query:         038.jpg | best score:  86 | predicted: 073
query:         039.jpg | best score: 101 | predicted: 073
query:         040.jpg | best score:  61 | predicted: 073
query:         041.jpg | best score: 213 | predicted: 073
query:         042.jpg | best score:  26 | predicted: not_in_dataset
query:         043.jpg | best score:  44 | predicted: 073
query:         044.jpg | best score:  83 | predicted: 073
query:         045.jpg | best score: 178 | predicted: 073
```

```
query:         046.jpg | best score:  24 | predicted: not_in_dataset
query:         047.jpg | best score:  49 | predicted: 047
query:         048.jpg | best score:  27 | predicted: not_in_dataset
query:         049.jpg | best score: 356 | predicted: 073
query:         050.jpg | best score:  35 | predicted: 073
query:         051.jpg | best score: 149 | predicted: 073
query:         052.jpg | best score: 129 | predicted: 073
query:         053.jpg | best score: 102 | predicted: 073
query:         054.jpg | best score:  59 | predicted: 046
query:         055.jpg | best score: 105 | predicted: 073
query:         056.jpg | best score: 150 | predicted: 073
query:         057.jpg | best score:  52 | predicted: 073
query:         058.jpg | best score:  64 | predicted: 073
query:         059.jpg | best score: 103 | predicted: 073
query:         060.jpg | best score:  62 | predicted: 073
query:         061.jpg | best score:  55 | predicted: 073
query:         062.jpg | best score: 310 | predicted: 073
query:         063.jpg | best score: 187 | predicted: 063
query:         064.jpg | best score:  70 | predicted: 064
query:         065.jpg | best score: 128 | predicted: 073
query:         066.jpg | best score:  47 | predicted: 066
query:         067.jpg | best score:  85 | predicted: 067
query:         068.jpg | best score:  46 | predicted: 073
query:         069.jpg | best score:  79 | predicted: 075
query:         070.jpg | best score:  17 | predicted: not_in_dataset
query:         071.jpg | best score: 122 | predicted: 071
query:         072.jpg | best score:  52 | predicted: 073
query:         073.jpg | best score:  31 | predicted: 073
query:         074.jpg | best score:  85 | predicted: 073
query:         075.jpg | best score: 262 | predicted: 075
query:         076.jpg | best score:  74 | predicted: 073
query:         077.jpg | best score:  94 | predicted: 073
query:         078.jpg | best score: 375 | predicted: 078
query:         079.jpg | best score: 182 | predicted: 079
query:         080.jpg | best score:  25 | predicted: not_in_dataset
query:         081.jpg | best score: 187 | predicted: 081
query:         082.jpg | best score:  94 | predicted: 082
query:         083.jpg | best score: 137 | predicted: 083
query:         084.jpg | best score: 163 | predicted: 084
query:         085.jpg | best score: 171 | predicted: 085
query:         086.jpg | best score:   8 | predicted: not_in_dataset
query:         087.jpg | best score:  69 | predicted: 087
query:         088.jpg | best score: 332 | predicted: 088
query:         089.jpg | best score:  75 | predicted: 089
query:         090.jpg | best score:  89 | predicted: 090
query:         091.jpg | best score:  32 | predicted: 091

accuracy on unknown queries: 12.09% (11/91 correctly rejected)
```

Your description of what you have done, and explanation of results, here

2.1 Question 3: FUndametal Matrix, Epilines and Retrival (optional, assesed for granting up to 25% bonus marks for the A2)

In this question, the aim is to accuractely estimate the fundamental matrix given two views of a scene, visualise the corresponding epipolar lines and use the inlier count of fundametal matrix for retrival.

The steps are as follows:

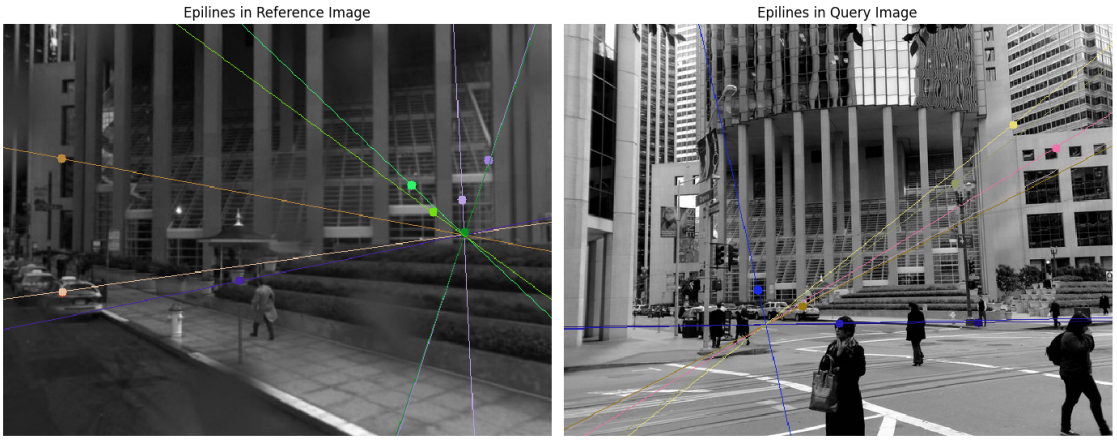
1. Select two images of the same scene (query and reference) from the landmark dataset and find the matches as you have done in Question 1 (1.1-1.4).
2. Compute fundametal metrix with good matches (after appying ratio test) using the opencv function cv.findFundamentalMat(). Use both 8 point algorithm and RANSAC assisted 8 point algorithm to compute fundamental matrix.
3. Hint: You need minimum 8 matches to be able to use the function. Ignore pairs where 8 mathes are not found.
4. Visualise the epipolar lines for the matched features and analyse the results. You can use openCV function cv.computeCorrespondEpilines() to estimate the epilines. We have provided the code for drawing these epilines in function drawlines() that you can modify as required.

```
good matches: 20

fundamental matrix (8-point):
[[ 1.66226819e-06 -3.40453895e-06  1.66579388e-04]
 [ 4.98763002e-06  1.85669520e-05 -6.48590505e-03]
 [-1.40487593e-03 -1.82284454e-03  1.00000000e+00]]

fundamental matrix (RANSAC):
[[ 2.14541910e-06  6.95108075e-07 -1.32450240e-03]
 [ 3.81136876e-06 -3.58664216e-07 -1.96070256e-03]
 [-1.84271739e-03 -3.79884212e-05  1.00000000e+00]]

RANSAC inliers: 10 / 20
```

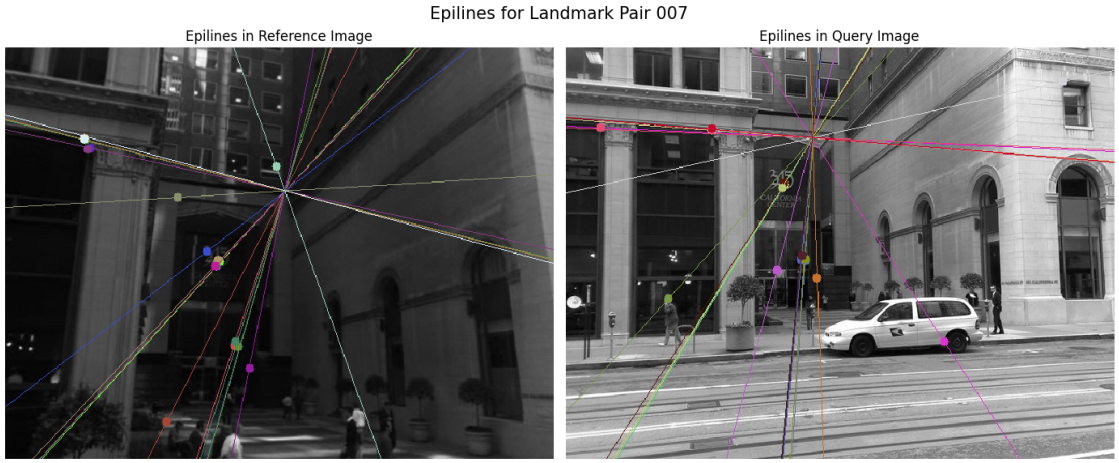


⌒ Your visualization for epilines goes here ⌒

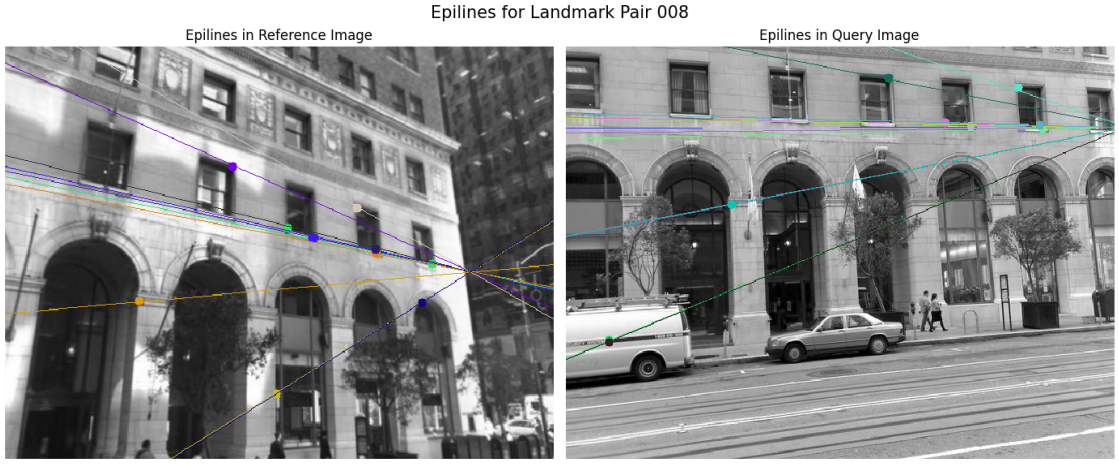
---

4. Repeat the steps for some examples from the landmarks datasets.

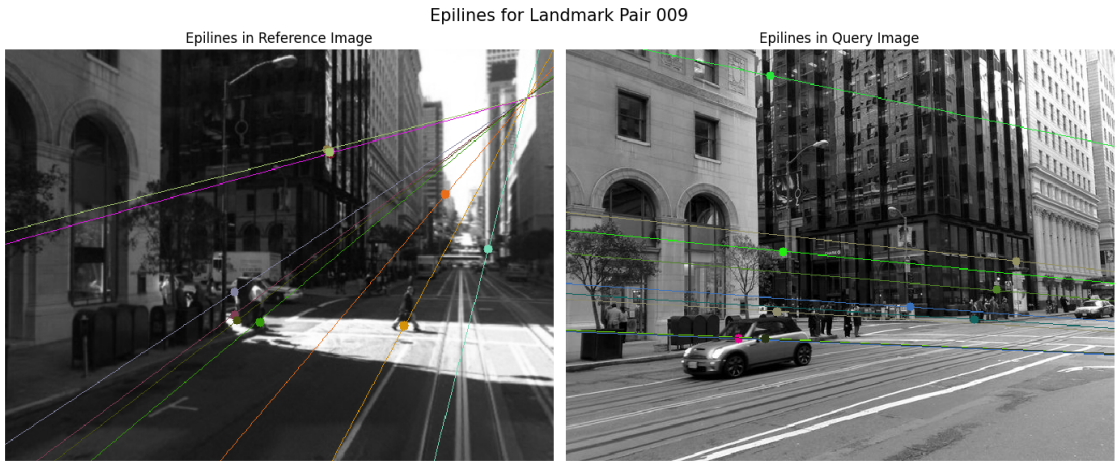
```
Image Pair 007
Good matches: 44
Fundamental Matrix (RANSAC):
[[ 2.21097769e-05 -1.30700137e-04  1.47237163e-02]
 [ 1.35032578e-04  3.11789902e-05 -4.92838000e-02]
 [-2.06993607e-02  3.42748192e-02  1.00000000e+00]]
Inliers after RANSAC: 20 / 44
```




```
Image Pair 008
Good matches: 26
Fundamental Matrix (RANSAC):
[[-5.44714892e-06  2.51281670e-05 -3.63826690e-03]
 [-3.75519425e-05  2.21051630e-05  1.46234724e-02]
 [ 7.12485144e-03 -1.85607323e-02  1.00000000e+00]]
Inliers after RANSAC: 13 / 26
```

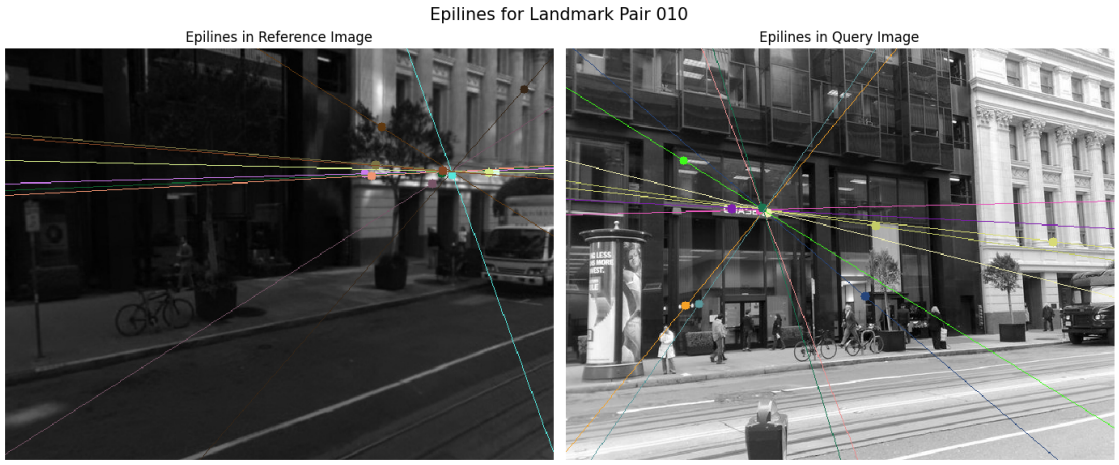



```
Image Pair 009
Good matches: 22
Fundamental Matrix (RANSAC):
[[-2.27915626e-07 -4.31204849e-07  1.63597942e-04]
 [ 4.58117722e-06  7.33755913e-06 -3.21105838e-03]
 [-1.43834691e-03 -2.16319990e-03  1.00000000e+00]]
Inliers after RANSAC: 11 / 22
```

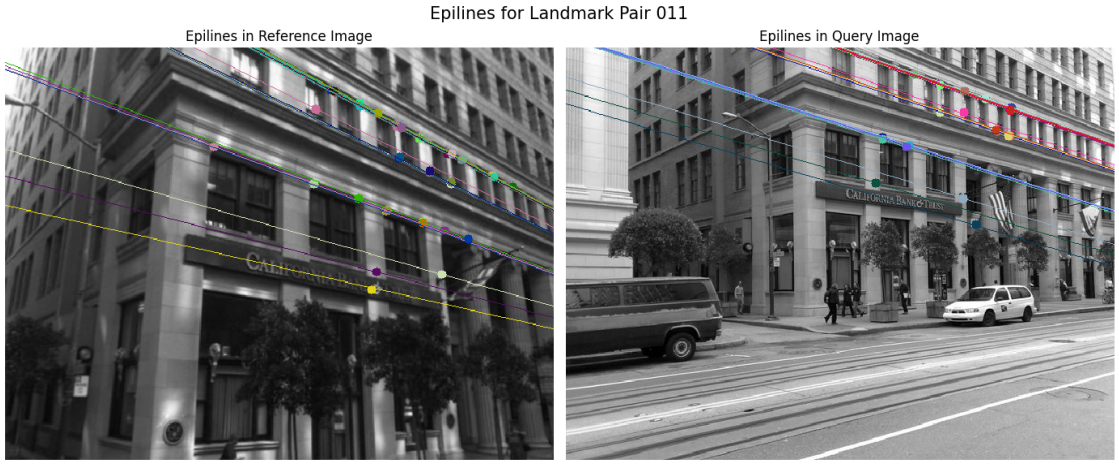



```
Image Pair 010
Good matches: 22
Fundamental Matrix (RANSAC):
[[ 2.01987931e-06 -2.33477972e-06 -7.15564680e-04]
 [-1.47698682e-06  3.60156522e-05 -4.42091417e-03]
 [-1.86648061e-04 -6.26460624e-03  1.00000000e+00]]
```

```
Inliers after RANSAC: 11 / 22
```




```
Image Pair 011
Good matches: 84
Fundamental Matrix (RANSAC):
[[ 1.19461283e-05  1.97145625e-06 -1.36560710e-02]
 [-4.55477947e-05  1.55912525e-05  4.22470942e-02]
 [ 7.55605048e-03 -2.14310684e-02  1.00000000e+00]]
Inliers after RANSAC: 38 / 84
```



^^ Your visualization for additional epillines goes here ^^

---

5. Find a query from landmarks data for which the retrival in Q2 failed. Attempt the retrival with replacing the Homography + RANSAC method of Q2 to Fundamental Matrix +

RANSAC method using code written above. Does the change of the model makes retraival successful? Analyse and comment.

```
Query: 002.jpg | Predicted: 0000 | Inliers: 19
```

Your analysis goes here

The goal was to estimate the **fundamental matrix** between two images of the same scene and analyze its effectiveness for retrieval, geometry, and visual interpretation.

3.1 Estimating the Fundamental Matrix I selected a pair of images (`urban_landmarks/001.jpg`) and: - Detected features using ORB, - Matched descriptors using a BFMatcher with Lowe's ratio test, - Estimated the **fundamental matrix** using two methods: - The **8-point algorithm** (pure least-squares), - **RANSAC**-assisted 8-point algorithm.

The RANSAC variant filtered outliers and retained 10 inliers out of 20 matches. Both matrices were valid, but the RANSAC estimate provided a better geometric fit, especially when visualizing the corresponding epipolar lines.

3.2 Epipolar Line Visualization Using `cv2.computeCorrespondEpilines`, I rendered epipolar lines in both reference and query images for the RANSAC inliers. The lines: - Correctly passed through (or near) their matching points, - Intersected consistent structural features like building edges and vanishing points, - Confirmed the estimated fundamental matrix captured the scene's epipolar geometry.

This visualization was essential for verifying correspondence accuracy and understanding stereo relationships.

3.3 Extension to Multiple Pairs I applied the same pipeline to five more image pairs: `urban_landmarks/007` through `011`. In each case, I: - Extracted keypoints and matches, - Estimated the fundamental matrix with RANSAC, - Visualized the resulting epilines.

Pair	Matches	Inliers	Notes
007	44	20	Accurate lines across vertical walls and windows.
008	26	13	Lines followed architectural features well.
009	22	11	Clean correspondences across parallel building faces.
010	22	11	Robust despite strong perspective change.
011	84	38	Very accurate — epilines tightly aligned with windows.

This showed the method's reliability across a range of views, even with challenging conditions (repetitive textures, occlusions, angle changes).

3.4 Comparison to Retrieval (Q2) Retrieval in Q2 used the number of **inliers from homography + RANSAC** as a score. While this worked well for planar scenes (like book covers), it struggled with non-planar scenes like urban landmarks.

To test model impact, I selected a **failed retrieval case** (`urban_landmarks/Query/002.jpg`). I replaced the homography scoring in Q2 with the **fundamental matrix + RANSAC** inlier count from Q3. The result:

- **Predicted:** `0000.jpg` (incorrect)
- **Inliers:** 19

The retrieval still failed, indicating that the failure was not just due to the geometric model but possibly: - Repeated textures in architecture, - Poor keypoint localization, - Ambiguities in perspective.

Fundamental matrix estimation is more general than homography, but still limited by the quality of input matches.

2.1.1 Final Thoughts

Question 3 provided valuable insight into: - How geometric models validate matches, - The strengths of epipolar geometry for visual understanding, - The limits of retrieval systems based purely on geometric inliers.

While fundamental matrix estimation is crucial for understanding stereo geometry, retrieval performance remains heavily dependent on keypoint quality, descriptor distinctiveness, and scene complexity. This highlights the need for integrating geometric methods with robust descriptors, ranking strategies, or learning-based retrieval in real-world applications.